

Types de données simples

Nombres (int, float)

exemples: `1 3.2 -1/3`

opérations de base: `+ - * /`

puissance: `**`

parenthèses contrôlent l'ordre des opérations:

`(1+2)*3`

division entière et reste: `// %`

arrondi et troncature: `round int`

fonctions math basiques: `max min abs`

Booléens (bool)

Deux valeurs possibles: `True False`

Chaînes de caractères (str)

chaîne: `"bonjour"`

chaîne vide: `""`

concaténation de chaînes:

`"bonjour " + "tout_le_monde"`

longueur(nombre de caractères):

`len("bonjour")` (donne 7)

Conversions entre types

`int() float() str()`

exemples: `x=int("42") s=str(42)`

Variables

un nom de variable commence par une lettre et ne contient pas d'espace.

assignation simple: `ma_variable=234.5`

assignation avec des expressions à droite:

`a=b+c x=x+1`

Afficher et Demander

afficher: `print(valeur1, valeur2, ...)`

demander: `réponse=input("question")`

demander et convertir en une fois:

`nb_entier=int(input("un nombre stp"))`

Commentaires

en ligne: `print("hello") # affiche hello`

bloc:

`"""`

`le code en-dessous
est génial`

`"""`

`print(42)`

Structures de contrôle

Conditions

simples: `== != < <= > >= in`

combinées: `and or not a < b < c`

Branchements conditionnels if / else

```
if condition:
    instructions si condition vraie
...
else: # optionnel
    instructions si condition fausse
...
instructions inconditionnelles
...
```

Boucles while

```
while condition:
    instructions qui se répètent
...
instructions en dehors de la boucle
...
```

Listes

liste: `L=[-1, 3.2, 1/3, "bob"]`

liste vide: `[]`

crochets clavier mac: `Alt+5 Alt+6`

récupérer et enlever un elt de la fin de la liste:

`dernier_element=L.pop()`

récupérer et enlever un elt du début de la liste:

`premier_element=L.pop(0)`

ajouter x à la fin: `L.append(x)`

insérer x à la position i: `L.insert(i, x)`

insérer x au début: `L.insert(0, x)`

savoir si une liste contient des éléments:

```
if L:
    instructions
...
```

itérer les éléments d'une liste du début à la fin:

```
while L:
    element=L.pop(0)
    instructions qui utilisent element
...
```

savoir si un élément est dans une liste:

```
if element in L:
    instructions
...
```

affectation de liste: `L1=L # même objet!`

copie de liste: `L1=L.copy()`

fonctions de liste: `max min len sum`